

VOOG: Real-Time Virtual Analog Synthesis in Python

A Hackable Instrument Architecture for Computer Musicians and DSP Prototypers

G. Pasquero

Independent Researcher and Developer
github.com/gpasquero/voog

Abstract—VOOG is a real-time polyphonic virtual analog synthesizer implemented in Python. It combines wavetable oscillators, noise generation, a 24 dB/oct Moog-inspired ladder filter, ADSR envelopes, LFO modulation, MIDI interaction, preset management, and a hardware-inspired graphical interface. The project is framed here not as a finished commercial synthesizer, but as a research instrument: a transparent and hackable environment for testing whether high-level languages can support playable subtractive synthesis without giving up readable DSP structure. We formalize the main signal model, oscillator phase accumulation, envelope state machine, nonlinear ladder filter approximation, modulation routing, and voice allocation strategy. The central claim is that Python can be musically useful for real-time virtual analog synthesis when computation is organized as block processing, stateful DSP units, and vectorized numerical kernels.

Index Terms—virtual analog synthesis, Python, real-time audio, Moog ladder filter, wavetable synthesis, computer music, DSP prototyping, MIDI

I. INTRODUCTION

Real-time audio software is usually associated with C, C++, Rust, or specialized environments such as SuperCollider and Pure Data. The reason is practical: interactive audio leaves little room for unpredictable latency, allocation spikes, or inefficient per-sample interpreter overhead. At the same time, many computer musicians and researchers prototype in high-level languages because they are fast to write, easy to inspect, and friendly to experimentation.

VOOG tests a provocative but useful question: *how far can a high-level language be pushed before it stops being an instrument?* The system implements a Moog-style subtractive synthesizer in Python with GUI controls, MIDI input, patch storage, 4 multitimbral channels, 8 voice polyphony per channel, 3 oscillators per voice, white/pink noise, glide, ADSR envelopes, LFO modulation, and a 24 dB/oct ladder filter [5]. The public repository describes the implementation as Python 3.13+, using NumPy and sounddevice for real-time output, optional mido/python-rtmidi for MIDI, and tkinter for the interface [5].

The contribution of this paper is threefold. First, it gives VOOG a formal DSP description suitable for technical discussion. Second, it identifies the algorithms and trade-offs that make the implementation musically playable. Third, it positions VOOG as a bridge between instrument building and algorithmic DSP prototyping.

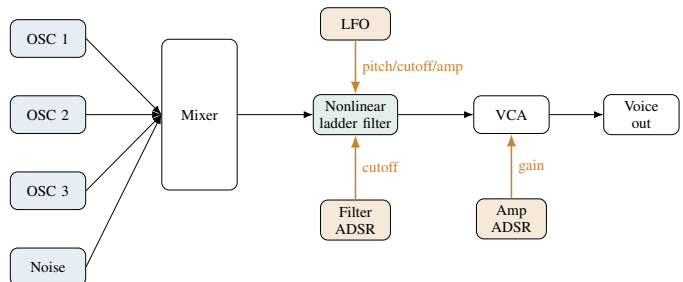


Fig. 1. Single-voice VOOG signal flow. The design follows classic subtractive synthesis while exposing every major DSP block as a replaceable software module.

II. DESIGN GOALS

VOOG is designed around four goals:

- 1) **Playability**: the program must respond as an instrument, not just render offline audio.
- 2) **Hackability**: the synthesis graph should remain understandable at source-code level.
- 3) **Virtual analog behavior**: the architecture should follow the subtractive grammar of classic analog synthesizers.
- 4) **Pedagogical visibility**: oscillators, envelopes, filters, modulation, voices, patches, and MIDI should remain separable modules.

This differs from plugin-first engineering, where optimized binary performance often hides the algorithmic structure. VOOG instead treats readability and modification speed as part of the musical affordance.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the voice-level signal flow. Each voice contains three oscillators and a noise source. These are mixed, passed through a nonlinear ladder filter, and multiplied by the amplitude envelope. A second envelope can modulate filter cutoff, while an LFO can target pitch, cutoff, or amplitude.

At a higher level, VOOG separates signal processing, engine logic, GUI, MIDI, and patch management. This separation matters because it lets musicians modify synthesis behavior without rewriting audio I/O or interface code.

IV. MATHEMATICAL SIGNAL MODEL

For a single voice, the synthesized output is modeled as

$$y[n] = a[n] \mathcal{F} \left(\sum_{m=1}^M g_m x_m[n] + g_N \eta[n]; c[n], r[n] \right), \quad (1)$$

where $x_m[n]$ is oscillator m , g_m is oscillator gain, $\eta[n]$ is noise, g_N is noise gain, $\mathcal{F}$ is the nonlinear filter, $a[n]$ is the amplitude envelope, $c[n]$ is the time-varying cutoff, and $r[n]$ is resonance.

For V active voices, the channel output is

$$s[n] = \sum_{v=1}^V y_v[n]. \quad (2)$$

A master gain G and optional limiter/safety scaling can then produce the final output

$$z[n] = G s[n]. \quad (3)$$

V. OSCILLATORS AND PHASE ACCUMULATION

Each oscillator is represented by a normalized phase $\phi[n] \in [0, 1)$:

$$\phi[n] = \left(\phi[n-1] + \frac{f[n]}{f_s} \right) \bmod 1, \quad (4)$$

where f_s is sample rate and $f[n]$ is the instantaneous oscillator frequency. The oscillator output is

$$x[n] = w(\phi[n]), \quad (5)$$

where w is a waveform function or wavetable lookup. Typical analytic waveforms include

$$w_{\text{saw}}(\phi) = 2\phi - 1, \quad (6)$$

$$w_{\text{square}}(\phi) = \begin{cases} 1, & \phi < d, \\ -1, & \phi \geq d, \end{cases} \quad (7)$$

$$w_{\text{tri}}(\phi) = 4 \left| \phi - \frac{1}{2} \right| - 1. \quad (8)$$

Here d is pulse width. In a wavetable implementation, the value is read from a table W :

$$x[n] = (1 - \alpha) W[k] + \alpha W[k+1], \quad (9)$$

with $k = \lfloor L\phi[n] \rfloor$, $\alpha = L\phi[n] - k$, and table length L .

A. Pitch, detune, and glide

MIDI note q maps to frequency

$$f(q) = 440 \cdot 2^{(q-69)/12}. \quad (10)$$

Octave, semitone, and cent detuning can be combined as

$$f_m = f(q) \cdot 2^{o_m} \cdot 2^{s_m/12} \cdot 2^{\delta_m/1200}. \quad (11)$$

For glide, the frequency can be smoothed using a one-pole transition

$$f[n] = f[n-1] + \beta (f_{\text{target}} - f[n-1]), \quad (12)$$

where $\beta = 1 - e^{-1/(\tau_g f_s)}$ and τ_g is the glide time constant.

B. Aliasing trade-off

Discontinuous waveforms generate harmonics above Nyquist. In a minimal wavetable or analytic oscillator, these harmonics fold back as aliasing. This is not ignored: it is a deliberate engineering trade-off that favors a readable and responsive system. Future versions should implement PolyBLEP, min-BLEP, or band-limited wavetable sets to reduce foldover while preserving the current architecture.

VI. NOISE GENERATOR

White noise is modeled as

$$\eta_w[n] \sim \mathcal{U}(-1, 1) \quad (13)$$

or as a zero-mean normal variate. Pink noise can be approximated by filtering white noise using a low-cost recursive filter. A common implementation is

$$\eta_p[n] = \sum_{i=1}^K b_i[n], \quad (14)$$

where each b_i is a slow random or filtered component. Pink noise is musically useful because it gives filtered sweeps and percussion patches a less brittle spectrum than white noise.

VII. ADSR ENVELOPES AS STATE MACHINES

The ADSR envelope is implemented as a state machine rather than a single closed-form function. Let $e[n]$ be the envelope value. For a target value T and time constant τ , a numerically stable exponential segment is

$$e[n] = e[n-1] + \gamma(T - e[n-1]), \quad \gamma = 1 - e^{-1/(\tau f_s)}. \quad (15)$$

The states are attack, decay, sustain, and release:

$$\text{attack : } T = 1, \quad (16)$$

$$\text{decay : } T = S, \quad (17)$$

$$\text{sustain : } e[n] = S, \quad (18)$$

$$\text{release : } T = 0. \quad (19)$$

The state transitions are triggered by note-on and note-off events. Exponential envelopes are preferred over purely linear envelopes because they better match perceptual loudness and analog RC behavior.

VIII. LFO AND MODULATION ROUTING

An LFO is another oscillator running at sub-audio rate:

$$\ell[n] = w_L(\phi_L[n]), \quad \phi_L[n] = \left(\phi_L[n-1] + \frac{f_L}{f_s} \right) \bmod 1. \quad (20)$$

A modulated parameter $p[n]$ is computed as

$$p[n] = p_0 + D_p \ell[n], \quad (21)$$

where p_0 is base value and D_p is modulation depth. For pitch modulation, a cents-domain representation is perceptually cleaner:

$$f[n] = f_0 \cdot 2^{D_c \ell[n]/1200}. \quad (22)$$

For filter cutoff, modulation is clamped to a safe range:

$$c[n] = \text{clip}(c_0 + D_c \ell[n] + D_e e_f[n], c_{\min}, c_{\max}). \quad (23)$$

Input: state, note event, attack A , decay D , sustain S , release R , current value e

```

if note-on then
  | state  $\leftarrow$  attack
end
if note-off then
  | state  $\leftarrow$  release
end
if state = attack then
  | update  $e$  toward 1 with  $\gamma_A$ ; if  $e \approx 1$ , state  $\leftarrow$  decay
else if state = decay then
  | update  $e$  toward  $S$  with  $\gamma_D$ ; if  $e \approx S$ , state  $\leftarrow$  sustain
else if state = sustain then
  |  $e \leftarrow S$ 
else if state = release then
  | update  $e$  toward 0 with  $\gamma_R$ ; if  $e \approx 0$ , mark voice inactive
end
return  $e$ , state

```

Algorithm 1: ADSR envelope update

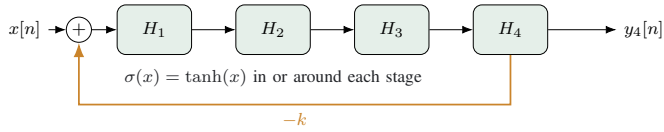


Fig. 2. Simplified four-stage nonlinear ladder filter with resonant feedback.

IX. NONLINEAR LADDER FILTER

The Moog ladder filter is one of the main sound-defining components of virtual analog synthesis. Analog ladder filters are nonlinear, resonant, and sensitive to feedback. Digital implementations have a long history, including analysis by Stilson and Smith and nonlinear approximations by Huovilainen [2], [1], [3].

A simplified discrete four-stage cascade can be written as

$$u[n] = x[n] - ky_4[n-1], \quad (24)$$

$$y_1[n] = y_1[n-1] + g(\sigma(u[n]) - \sigma(y_1[n-1])), \quad (25)$$

$$y_2[n] = y_2[n-1] + g(\sigma(y_1[n]) - \sigma(y_2[n-1])), \quad (26)$$

$$y_3[n] = y_3[n-1] + g(\sigma(y_2[n]) - \sigma(y_3[n-1])), \quad (27)$$

$$y_4[n] = y_4[n-1] + g(\sigma(y_3[n]) - \sigma(y_4[n-1])), \quad (28)$$

where k controls resonance, g is related to cutoff, and $\sigma(\cdot)$ is a saturating nonlinearity. A common choice is

$$\sigma(x) = \tanh(x). \quad (29)$$

Fig. 2 illustrates the feedback topology. The important musical result is that resonance and saturation are not added as cosmetic effects; they are part of the recursive state of the filter.

The coefficient g can be derived from a one-pole low-pass discretization. A simple choice is

$$g = 1 - e^{-2\pi c/f_s}, \quad (30)$$

Input: note q , velocity v , voice pool P

```

if there exists inactive voice  $p \in P$  then
  | assign  $q$  to  $p$ ; reset phase/envelopes as needed;
  | return
end
 $R \leftarrow$  voices currently in release state
if  $R$  is not empty then
  | steal quietest voice in  $R$ 
end
else
  | steal oldest active voice or lowest-amplitude voice
end
initialize stolen voice with note  $q$  and velocity  $v$ 

```

Algorithm 2: Polyphonic voice allocation with stealing

where c is cutoff frequency. More accurate topologies can use trapezoidal integration or zero-delay feedback, but the cascade model remains attractive because it is readable, cheap, and musically convincing.

X. VOICE ALLOCATION

Polyphony converts the synthesizer from a single DSP chain into a scheduling problem. A voice contains oscillator phases, envelope states, filter memories, note identity, velocity, and age. On note-on, the allocator searches for an inactive voice. If none is available, it steals a voice according to a musical priority rule.

This is not merely housekeeping. Voice stealing has an audible signature. Stealing the oldest voice can be predictable but musically abrupt. Stealing the quietest release-phase voice usually minimizes discontinuities.

XI. BLOCK PROCESSING AND PYTHON FEASIBILITY

A naive Python implementation that calls Python code for every operation of every sample is unlikely to satisfy real-time constraints. VOOG's feasibility comes from block processing and numerical vectorization. Given block size B , a voice computes vectors

$$\mathbf{y}_v = \mathbf{a}_v \odot \mathcal{F}(\mathbf{x}_v), \quad \mathbf{x}_v \in \mathbb{R}^B. \quad (31)$$

The channel mix is

$$\mathbf{s} = \sum_{v=1}^V \mathbf{y}_v. \quad (32)$$

The per-block computational cost can be described as

$$\mathcal{O}(BV(M + C_F + C_E + C_M)), \quad (33)$$

where M is oscillator count, C_F is filter cost, C_E is envelope cost, and C_M is modulation cost. The critical design principle is to keep the Python control layer at event and block granularity, while NumPy handles contiguous numerical work.

TABLE I

BENCHMARK TEMPLATE FOR REPRODUCIBLE PUBLICATION. VALUES MUST BE MEASURED ON A DECLARED MACHINE AND AUDIO INTERFACE.

Patch	Voices	CPU %	Underruns/min	Latency ms
Init	1	TBD	TBD	TBD
Bass Voog	4	TBD	TBD	TBD
Pad Strings	8	TBD	TBD	TBD
Trance Lead	16	TBD	TBD	TBD
Noise Sweep	32	TBD	TBD	TBD

XII. EVALUATION METHODOLOGY

This draft does not invent benchmark numbers. For publication, VOOG should be evaluated with a reproducible protocol:

- 1) fixed sample rate, buffer size, and host machine;
- 2) patches grouped by oscillator count, filter resonance, and modulation depth;
- 3) sustained chords at 1, 4, 8, 16, and 32 voices;
- 4) measured callback underruns, CPU load, and end-to-end latency;
- 5) comparison against equivalent patches in SuperCollider, Pure Data, and a JUCE reference implementation.

Table I shows a benchmark table template. Numbers should be filled from measured runs only.

XIII. RELATED WORK

SuperCollider provides a mature language and server model for real-time synthesis [6]. Pure Data provides a visual patching environment that remains central to experimental music systems [7]. JUCE is widely used for professional audio applications and plugins [8]. VOOG does not attempt to replace these ecosystems. Its niche is different: it is a compact Python instrument that keeps synthesis algorithms close to the musical code, making it attractive for teaching, hacking, and rapid modification.

In virtual analog synthesis, oscillator aliasing and nonlinear filter behavior are central problems [3]. VOOG currently emphasizes architectural clarity and playability; future work should improve oscillator band-limiting and filter numerical accuracy while preserving accessibility.

XIV. DISCUSSION

The strongest argument for VOOG is not that Python is universally fast enough. It is that real-time audio systems contain multiple time scales. Audio samples require strict timing, but user gestures, MIDI events, patch changes, GUI movement, and modulation routing can often be managed at slower control rates or block boundaries. A well-designed Python synthesizer exploits this separation.

For computer musicians, the result is important: the instrument is not a black box. Its sound can be changed by editing the algorithms directly. This makes VOOG a useful platform for workshops, sound design research, pedagogical DSP, and experimental performance systems.

XV. LIMITATIONS AND FUTURE WORK

The current architecture has limitations that should be stated directly:

- analytic or simple wavetable oscillators can alias;
- Python scheduling and garbage collection can affect worst-case timing;
- tkinter is not a professional plugin UI framework;
- no VST3/AU plugin wrapper is currently described;
- benchmark data should be collected before formal submission.

Future work should add PolyBLEP or band-limited wavetable oscillators, zero-delay feedback filter variants, denormal protection, Cython/Numba or C++ acceleration for hot paths, plugin export, preset morphing, and a public audio example set.

XVI. CONCLUSION

VOOG demonstrates a compelling middle ground between DSP prototype and playable instrument. By implementing a virtual analog synthesizer in Python, it challenges the assumption that real-time audio must begin in low-level code. The project is most valuable as a hackable research instrument for computer musicians: readable enough to modify, structured enough to study, and immediate enough to play.

ACKNOWLEDGMENT

This paper draft is based on the public VOOG repository by G. Pasquero and is intended as a publication-oriented technical framing of the project.

REFERENCES

- [1] A. Huovilainen, "Non-Linear Digital Implementation of the Moog Ladder Filter," *Proc. International Conference on Digital Audio Effects (DAFx)*, 2004.
- [2] T. Stilson and J. O. Smith, "Analyzing the Moog VCF with Considerations for Digital Implementation," CCRMA, Stanford University, 1996.
- [3] V. Valimaki and A. Huovilainen, "Oscillator and Filter Algorithms for Virtual Analog Synthesis," *Computer Music Journal*, 2006.
- [4] J. O. Smith III, *Physical Audio Signal Processing*. W3K Publishing, 2010.
- [5] G. Pasquero, "VOOG: Virtual Analog Synthesizer," GitHub repository, 2026. Available: <https://github.com/gpasquero/voog>.
- [6] J. McCartney, "SuperCollider: A New Real Time Synthesis Language," *Proc. International Computer Music Conference*, 1996.
- [7] M. Puckette, "Pure Data: Another Integrated Computer Music Environment," 1996.
- [8] JUCE, "JUICE C++ framework for audio applications," 2026. Available: <https://juce.com/>.